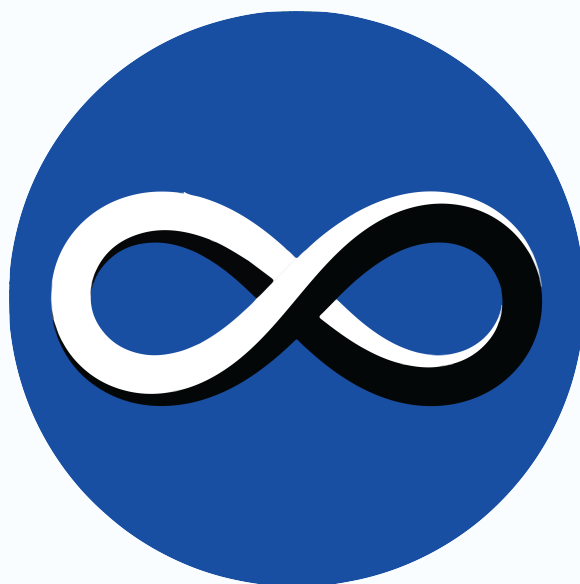


PROJECT MEMBER TASKS

OASIS

2026-7



MATHEMATICS CLUB



CENTRE FOR INNOVATION
IIT MADRAS

Week-1 tasks

1. Familiarize yourself with GitHub, Latex and Notion.
2. Please create a new folder in the following directory:

Folder Path

```
.\Contributors\PM tasks\Your Name\
```

Example

```
.\Contributors\PM tasks\Mohit\
```

You're supposed to add the responses to the tasks in your folder and make a PR request.

3. Divide the topics from the second chapter of the ISLR book and present in the next meeting.
4. Setup **VS-Code** and **MinGW** which are essential to write and execute C++ code (MinGW is not needed if you're a Linux user).

Week-2 tasks

5. Learn how to write basic functions, loading a dataset and estimate few parameters in C++.
6. Take a dataset from [here](#) and load this and evaluate basic statistical parameters.
7. Read the **Simple** and **Multiple** Linear regression topics from ISLR book (Chapter-3). (sections 3.1 3.2 3.3)

Week-3 (To repeat the previous week, but in greater depth) tasks

8. Learn how to write basic functions, loading a dataset and estimate few parameters in C++. Take a dataset from [here](#) and load this and evaluate basic statistical parameters. A few directions are given below, but feel free to learn about anything else you want to.
 - `fstream`, `getline`
 - `std::stoi/std::stod`
 - `std::string_view`
 - `std::from_chars`
 - `std::fast_float`
 - Any other way to optimize dataset loading and conversion, be it better algorithm, better data structures or better code via parallelization or cache.

- Statistical Parameters: Explore Eigen3 documentation
9. Read the **Simple** and **Multiple** Linear regression topics from ISLR book (Chapter-3) [Upto 3.3] in better depth.
- How $SE(\hat{\beta})$ is estimated i.e. $SE(\hat{\beta})$ (the derivation) and what degree of freedom is.
 - Student's t-distribution, t-test, pairwise t-test.
 - Hypothesis test, p-value, α , "confidence", type-I/II errors.
 - F-statistic, the expectations in the book and its p-value
 - AIC, BIC, Adjusted R², Mallows' Cp - the tradeoffs between "model explanation and variable addition" that terms introduce in each of these statistics' formulae.
 - synergy effect
 - 3.3 - what if OLS assumptions fail. Learn the below.
 - Assumptions of Linear Models generally
 - Assumptions of OLS that are extra than linear models.
 - what if OLS assumptions fail.
 - If you're done with everything, get to know why OLS is BLUE.

Week-4 (Regularization and Sparse Models) tasks

10. Learn why Ordinary Least Squares is often not enough.
- High variance
 - Overfitting
 - Multicollinearity
 - Bias-Variance tradeoff
11. Read the Ridge Regression and Lasso sections from ISLR (Chapter 6).
- Ridge Regression
 - Objective function
 - Why the L_2 penalty shrinks coefficients
 - Effect of changing λ
 - Closed-form solution
 - Why do coefs never go to zero?
 - Lasso
 - Objective function

- Why the L_1 penalty produces sparse solutions
 - Feature selection
 - Geometric intuition (diamond/sphere)
- Compare
 - OLS vs Ridge vs Lasso
 - When to use which
 - Effect on bias and variance
- Learn how λ is selected.
 - Validation set
 - Cross-validation
 - One-standard-error rule

12. Coding

- Implement Ridge Regression from scratch in C++ using Eigen3.
- Implement Lasso using coordinate descent.
- Standardize features before training.
- Plot training and validation error as λ changes.
- Plot coefficient paths as λ varies.
- Compare Ridge and Lasso on at least one real dataset.

13. Read the `glmnet` paper.

- What problem is the paper trying to solve?
- Why is coordinate descent an effective optimization algorithm?
- What is a regularization path?
- What are warm starts and why do they make the algorithm faster?
- Understand the idea behind strong rules for discarding predictors.
- How is cross-validation used to select λ ?
- Relate the implementation ideas in the paper to your own implementation.

14. Programming Concepts that can help.

- Stack vs Heap memory.
- References, pointers and basic memory management.
- Pass-by-value vs pass-by-reference vs `const` reference.
- Constructors, destructors and object lifetime.
- Why unnecessary copying can make programs slow.

- `std::vector` internals.
 - Contiguous memory
 - `reserve()`
 - `resize()`
 - `push_back()` and reallocation
- Cache locality and why contiguous memory improves performance.
- Move semantics (basic intuition only).
- Reading modern C++ function signatures.

